Open in app ↗

Medium

Member-only story

How I Added Persistent Semantic Memory to Claude Code in 15 Minutes

Mem0 + MCP + ChromaDB — plus what happens when your “helpful” hook spawns seven parallel Claude instances

10 min read · 4 days ago

Daniel Rusnok Follow

Listen Share ••• More

Not a member? Read the entire article for free via this link.

I asked Claude Code why I chose Inngest over raw cron jobs for Drippery. It had no idea. I'd made that call weeks earlier in a different session, written nothing down, and now it was gone.

Claude Code is great at writing code. **It's terrible at remembering what I told it yesterday.**

Every new session starts from scratch. CLAUDE.md helps, but it's a flat file with no search — when I need "what did I decide about auth for the public API?", I'm grepping through a markdown file like it's 2005.

I wanted something that **captures decisions automatically and lets me search them in natural language** across all my projects.

> *Here's how I built it in 15 minutes with zero infrastructure — and how the same setup nearly overheated my MacBook Air M4 a week later.*

Join the Digital Craft Workshop

A 5-day welcome series + weekly essays on building digital products with care. No fluff, no hype - just craft.

drippery.app

The Problem: AI Amnesia

I work with Claude Code daily. Over weeks, I've accumulated dozens of architectural decisions for Drippery:

- Why I chose Inngest over raw cron jobs
- Why the embed system is decoupled from sequences
- Why does the public API use x-api-key instead of OAuth

- Why I went with Resend over SendGrid

Every new Claude Code session, I either re-explain this or hope the CLAUDE.md catches it. **It usually doesn't.**

```
# Drippery — project notes for Claude Code

## Stack
- Next.js 16 App Router, TypeScript strict mode
- PostgreSQL via Drizzle ORM (migrations in `/db/migrations`)
- Clerk for auth (multi-tenant, organization-based)
- Stripe for billing, Resend for transactional email, Inngest for background jobs
- Render for hosting (Frankfurt region), Cloudflare R2 for image uploads
- Deployed via `main` branch auto-deploy, preview envs on PR

## Decisions

- Chose Inngest over raw cron jobs because I need retries, backoff, and step functions without running my own worker. Cron is cheaper but loses state on restart. 2
- Inngest function names must be `kebab-case`. The dashboard treats them as stable IDs, renaming breaks history.
- Resend over SendGrid — pricing is simpler ($0 for 3k/mo, $20 for 50k), API is cleaner, and the React Email integration saved me a template engine. SendGrid would
scale but I'm not there.
- Webhook signing for Resend: using `svix` library, secret in `RESEND_WEBHOOK_SECRET`. Do NOT parse webhook body before verifying signature — tried it, broke it, s
- Clerk organizations = tenants. Users can belong to multiple orgs. The active org is in the session — always read via `auth().orgId`, never from URL or form.
- Decided against custom roles in Clerk — using Clerk's built-in `admin` / `basic_member`. If we need more granularity later, switch to Clerk RBAC + our own permiss
- Drizzle schema: all tables have `id` (uuid v7), `createdAt`, `updatedAt`, `orgId` (FK to organizations). No soft deletes — use audit log table instead.
- UUID v7 chosen for time-sortable IDs. Makes pagination work without cursor shenanigans. 2026-02-03.
- Public API uses `x-api-key` header instead of OAuth. Reason: target user is a creator pasting a key into their Substack or blog, OAuth flow is overkill and scary.
revocable from dashboard. 2026-02-20.
- API keys are stored hashed (sha256), prefix is visible in UI for identification (`dk_live_abc1_`). Never log the full key.
- Rate limit: 100 req/min per key for free, 1000 req/min for paid. Redis-backed sliding window. Limits in `/lib/ratelimit.ts`.
- Embed widget is decoupled from sequences — it's a separate `embeds` table with a foreign key to a sequence. Reason: one sequence can be embedded in multiple plac
analytics. 2026-02-27.
- Embed widget ships as a single `<script>` tag that injects a shadow-DOM form. No iframe — iframes get blocked on Substack.
- Embed CSS uses only CSS variables + shadow DOM so it can't be styled by the host page. Tested on Substack, Ghost, WordPress, raw HTML.
- Confirmation emails are double opt-in by default, single opt-in is a Pro-only feature. GDPR needs this and I'm not fighting that fight.
- Unsubscribe link is REQUIRED in every send. Injected server-side, can't be overridden by template. If user tries to remove `{{unsubscribe_url}}`, the send API re
- Unsubscribe is one-click (CAN-SPAM compliant), doesn't require a confirmation page. Tracking pixel on open, link wrapper on click — standard email analytics.
- Bounce handling: Resend webhooks → Inngest → mark subscriber as `bounced` in DB. Hard bounces get suppressed globally. Soft bounces retry 3x over 72h.
- Complaints (marked as spam) → immediate suppression + flag the sequence for review. Three complaints in 24h = freeze sequence, notify owner.
- Decided to NOT build visual sequence editor. Sequences are defined as a list of steps with delay + template. Much simpler UI, fewer bugs. Revisit if users ask.
- Step delays are stored as ISO 8601 durations (`P1D`, `PT2H`). Inngest handles the scheduling.
- Minimum step delay is 5 minutes. Reason: protects against accidental spam loops. Max delay is 90 days.
- Max sequence length is 20 steps on Pro, 5 on Starter, 1 on Free. Hardcoded in `/lib/plans.ts`.
```

wall of text in CLAUDE.md

Why not Claude-Mem?

Before building my own, I tried claude-mem — the most popular persistent memory plugin for Claude Code with 46K+ GitHub stars. It's purpose-built, well-maintained, and does exactly what I need.

But I hit critical bugs on Apple Silicon. At the time I tested it, the MCP loopback wasn't saving observations reliably, the worker daemon kept timing out, and there were Bun-related crashes on macOS. Multiple open issues, no fix in sight.

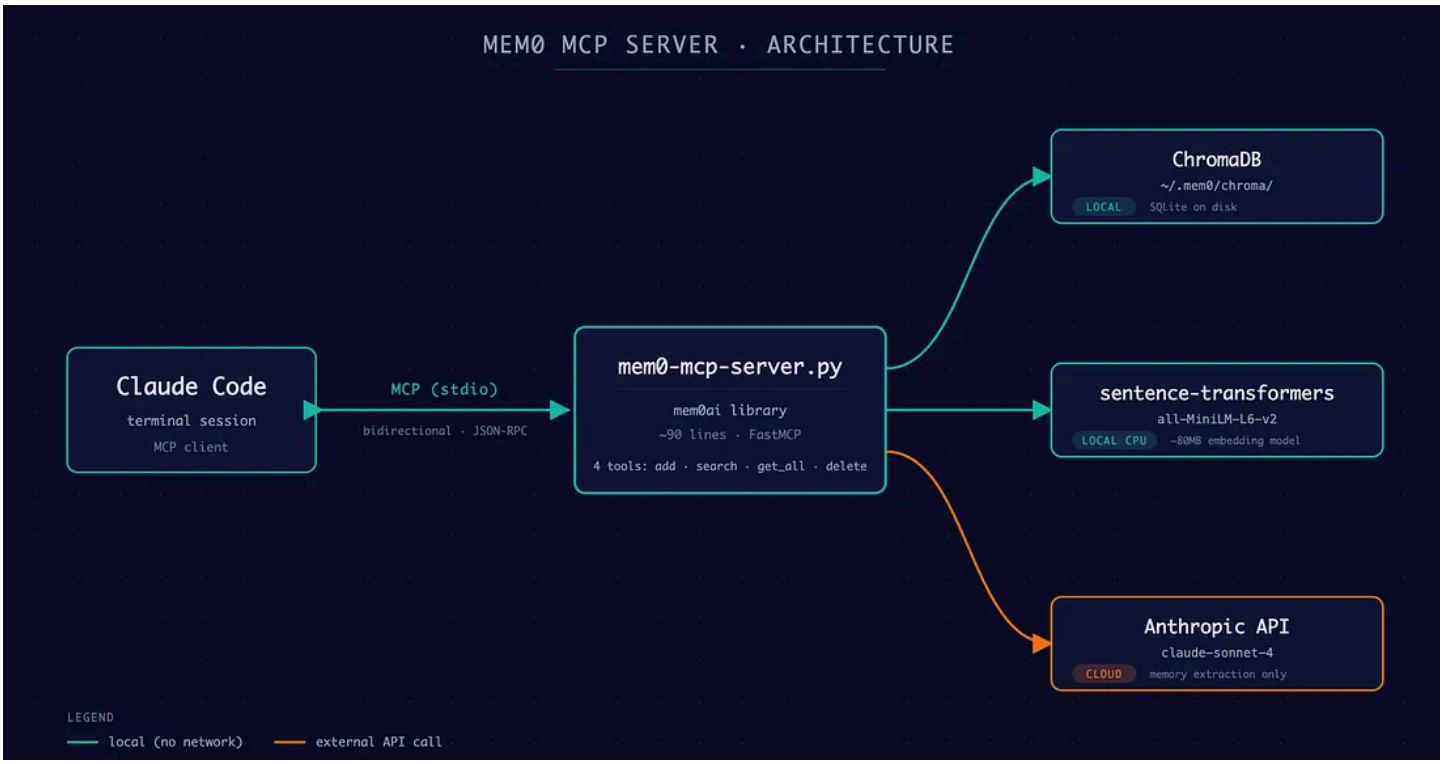
If you're on Intel Linux, it might work fine — on my M4 Mac, it was unusable. **So I built something simpler.** Fewer features, but it actually runs.

The Solution: Mem0 as an MCP Server

The fix is simple. **Three components:**

1. **Mem0** (mem0ai) — the memory layer. It takes text, extracts key facts using an LLM, stores embeddings in a vector database, and lets you semantic-search later.
2. **MCP (Model Context Protocol)** — the standard way to give Claude Code new tools. You write a server, and Claude gets new capabilities.
3. **ChromaDB** — local vector store. No Docker, no Qdrant, no PostgreSQL. Just a SQLite file on disk.

The architecture:



The architecture | Generated with Claude

Step 1: Install Dependencies

```
pip install mem0ai chromadb sentence-transformers mcp anthropic
```

That's it. **No Docker compose, no database setup, no config files.** sentence-transformers downloads the embedding model (~80MB) on the first run.

Step 2: Write the MCP Server

The entire server is ~90 lines of Python. Here's the core:

```python
#!/usr/bin/env python3
import os, sys, json
from mcp.server.fastmcp import FastMCP
CONFIG = {
    "llm": {
        "provider": "anthropic",
        "config": {
            "model": "claude-sonnet-4-20250514",
            "api_key": os.environ.get("ANTHROPIC_API_KEY", ""),
        },
    },
    "embedder": {
        "provider": "huggingface",
        "config": {"model": "all-MiniLM-L6-v2"},
    },
    "vector_store": {
        "provider": "chroma",
        "config": {
            "collection_name": "mem0",
            "path": os.path.expanduser("~/.mem0/chroma"),
        },
    },
}
_memory = None
def get_memory():
    global _memory
    if _memory is None:
        from mem0 import Memory
        _memory = Memory.from_config(CONFIG)
    return _memory

mcp = FastMCP("mem0")

@mcp.tool()
def mem0_add(text: str, user_id: str = "default", metadata: dict | None = None)
    """Store a memory — decisions, preferences, architecture choices."""
    result = get_memory().add(text, user_id=user_id, metadata=metadata or {})
    return json.dumps(result, default=str)

@mcp.tool()
def mem0_search(query: str, user_id: str = "default", limit: int = 5) -> str:
    """Semantic search across all stored memories."""
    results = get_memory().search(query, user_id=user_id, limit=limit)
    return json.dumps(results, default=str)
```

```python
@mcp.tool()
def mem0_get_all(user_id: str = "default") -> str:
    """List all stored memories for a user."""
    results = get_memory().get_all(user_id=user_id)
    return json.dumps(results, default=str)

@mcp.tool()
def mem0_delete(memory_id: str) -> str:
    """Delete a specific memory by its ID."""
    get_memory().delete(memory_id)
    return json.dumps({"deleted": memory_id})

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Save this as `~/.claude/mem0-mcp-server.py`.

Key design decisions:

- **Lazy loading** — the Memory object (and embedding model) loads on the first tool call, not on server start. This keeps the Claude Code startup fast.
- **HuggingFace embeddings** — all-MiniLM-L6-v2 runs locally, no API calls for embeddings. Only the memory extraction LLM hits Anthropic's API.
- **ChromaDB on disk** — data lives in `~/.mem0/chroma/`. Survives reboots, no server process needed.

Step 3: Register with Claude Code

```bash
claude mcp add mem0 -s user -- python3 ~/.claude/mem0-mcp-server.py
```

The `-s user` flag makes it available across all projects, not just the current one. **Restart Claude Code, and you'll see four new MCP tools.**

Step 4: Automate with a Hook

Manually calling "remember this" gets old fast. I wired a hook that runs a background `claude -p` instance on `SessionEnd` (and originally `PreCompact` too) — it reads the transcript, extracts non-obvious decisions, and stores them via `mem0_add`.

It worked. For about a week. Then my MacBook's fans ramped to max and wouldn't stop.

The Incident: Seven Parallel Claude Instances

One afternoon, my MacBook Air M4 — a machine that normally runs silently — **got uncomfortably hot.** The load average was 6.32 on a 10-core CPU.

I opened `top` and found the culprit:

```
claude -p "Extract key..."   72.9% CPU
claude -p "Extract key..."   66.1% CPU
claude -p "Extract key..."   44.9% CPU
claude -p "Extract key..."   36.8% CPU
claude -p "Extract key..."   25.0% CPU
claude -p "Extract key..."   19.1% CPU
claude -p "Extract key..."    —
```

Seven parallel headless Claude instances. Four `mem0-mcp-server.py` processes alongside them. All spawned by my own "helpful" memory hook.

```
daniel@macbook-air – top – 180×36

daniel@macbook-air ~ % top -o cpu -n 15
Processes: 647 total, 11 running, 636 sleeping, 3102 threads                    15:42:17
Load Avg: 6.32, 4.18, 2.75  CPU usage: 88.73% user, 11.27% sys, 0.00% idle
SharedLibs: 412M resident, 63M data, 30M linkedit.
MemRegions: 251837 total, 4287M resident, 178M private, 1523M shared.
PhysMem: 15G used (2857M wired, 1012M compressor), 221M unused.
VM: 172T vsize, 5214M framework vsize, 0(0) swapins, 0(0) swapouts.
Networks: packets: 1253147/1263M in, 798152/342M out.
Disks: 842512/19G read, 341247/8G written.

PID     COMMAND                %CPU   TIME      #TH  MEM     STATE       PPID
82341   claude -p "Extrahuj…"  72.9   00:04:12  47   984M    running     1
82156   claude -p "Extrahuj…"  66.1   00:03:58  44   912M    running     1
83102   claude -p "Extrahuj…"  44.9   00:02:14  39   856M    running     1
83245   claude -p "Extrahuj…"  36.8   00:01:49  37   821M    running     1
83891   claude -p "Extrahuj…"  25.0   00:00:58  34   754M    running     1
84012   claude -p "Extrahuj…"  19.1   00:00:32  32   712M    running     1
84187   claude -p "Extrahuj…"  –      00:00:02  15   124M    stuck       1
82350   python3 mem0-mcp-se…   12.4   00:01:23  8    342M    running     82341
82165   python3 mem0-mcp-se…   9.8    00:01:08  8    318M    running     82156
83110   python3 mem0-mcp-se…   6.2    00:00:47  7    286M    running     83102
83253   python3 mem0-mcp-se…   4.1    00:00:31  7    274M    running     83245
501     kernel_task            5.4    12:34:56  128  156M    running     0
234     WindowServer           2.1    03:21:45  18   284M    sleeping    1
1847    Electron Helper …      1.9    00:34:12  22   412M    sleeping    1842
1921    node                   1.2    00:12:08  9    186M    sleeping    1
```

Activity monitor | Generated with Claude

Root Cause: Four Bugs Stacked Together

I pulled the script apart and found **four issues:**

1. **Per-session lock, not global.** My concurrency guard was `/tmp/claude-mem0-locks/${SESSION_ID}.lock`. If five sessions ended at roughly the same time, each one had its own lock file — so each one happily spawned its own extraction. There was no global cap.

2. **PreCompact fires during sessions, not just at the end.** I had registered the same hook on both `SessionEnd` and `PreCompact`, assuming PreCompact was rare. It isn't. **It fires every time Claude Code compacts its context** — multiple times per long session.

3. **No hard timeout on the spawned process.** The `timeout: 10` in `settings.json` only applies to the shell wrapper. The wrapper returns instantly because it backgrounds the real work with `nohup ... &`. The actual `claude -p` instance? No limit. If it got stuck, it stayed stuck forever.

4. **No cleanup of orphaned MCP servers.** Each spawned `claude -p` started its own `mem0-mcp-server.py`. When the extraction hung, the Python server hung with it.

> *Any one of these on its own is survivable. All four together turn a memory hook into a runaway process.*

The Fix

I rewrote the hook with three changes:

1. Global concurrency cap via `pgrep`. Before spawning anything, check the entire process table:

```
RUNNING=$(pgrep -f "claude.*dangerously-skip-permissions.*Extrahuj klíčové" | w
if [ "${RUNNING:-0}" -ge 1 ]; then
    exit 0
fi
```

If any extraction is already running system-wide, skip. **One at a time, period.**

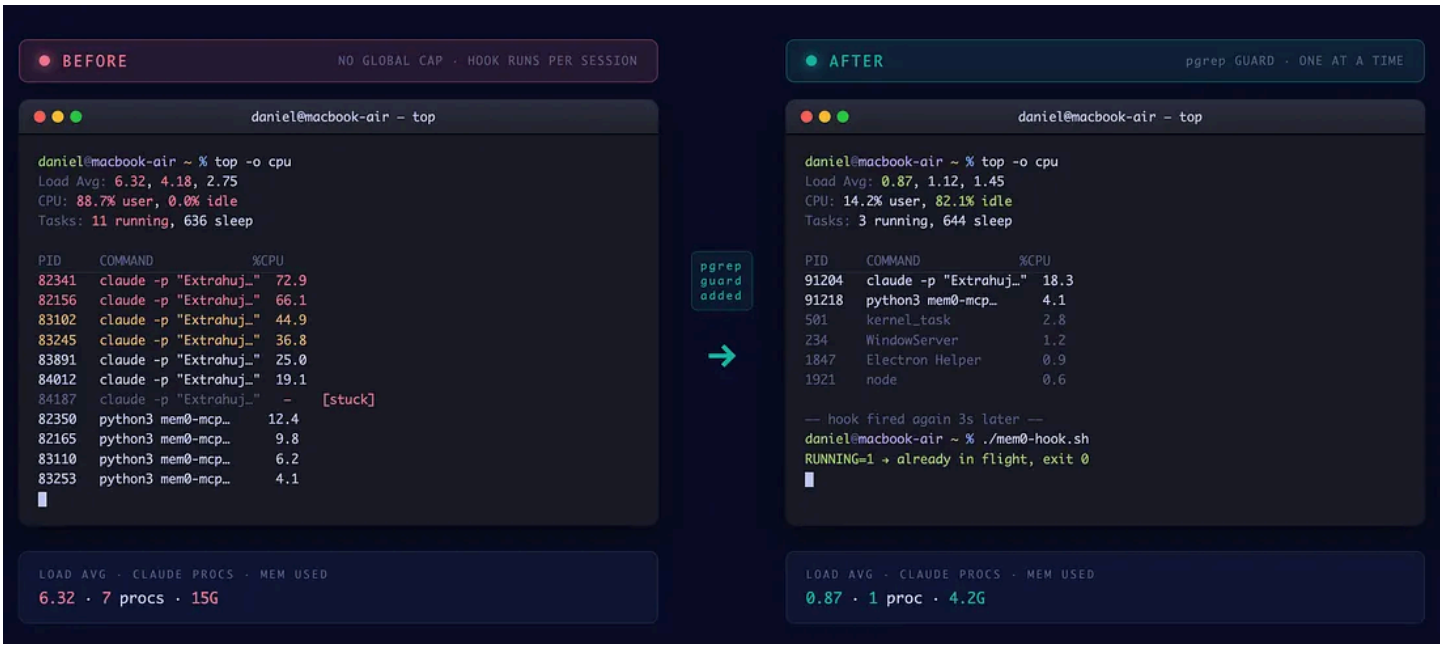
2. Remove the PreCompact hook entirely. `SessionEnd` alone is enough. If you want per-compact capture, you need your own throttle — don't just paste the same hook under both events.

3. Hard timeout on the spawned process. macOS doesn't ship `timeout` natively, so I detect `gtimeout` from coreutils:

```
TIMEOUT_BIN=""
for p in /usr/bin/timeout /opt/homebrew/bin/gtimeout /usr/local/bin/gtimeout; c
    if [ -x "$p" ]; then TIMEOUT_BIN="$p"; break; fi
done
nohup "$TIMEOUT_BIN" 300 "$CLAUDE_BIN" --dangerously-skip-permissions -p "$PROM
    > "$LOG_FILE" 2>&1 &
```

5-minute hard ceiling. If the extraction hangs, the OS kills it. Install with `brew install coreutils`.

After the fix, I re-tested by firing two hook calls back-to-back. Running instances after both fires: **1**. Global cap works.



Before/After Comparison | Generated by Claude

The Real Lesson

The original hook script wasn't written by me. **It was written by Claude Code, at my request, and I installed it without reading it carefully enough.** It worked on day one. It was broken by day eight — I just didn't know it.

The pattern is easy to miss: AI-generated infrastructure code looks correct, passes a dry-run, and **only fails under real-world concurrency that a single test can't reproduce.** These weren't syntax errors. They were wrong assumptions about the world:

- "Only one session ends at a time" — wrong, I often have 3+ Claude Code windows open.
- "PreCompact is rare" — wrong, it fires on every context compaction.

- “`timeout: 10` in settings.json contains the whole operation" — wrong, it only covers the wrapper.
- “nohup& is fire-and-forget and that’s fine” — wrong, fire-and-forget without cleanup is how runaway processes start.

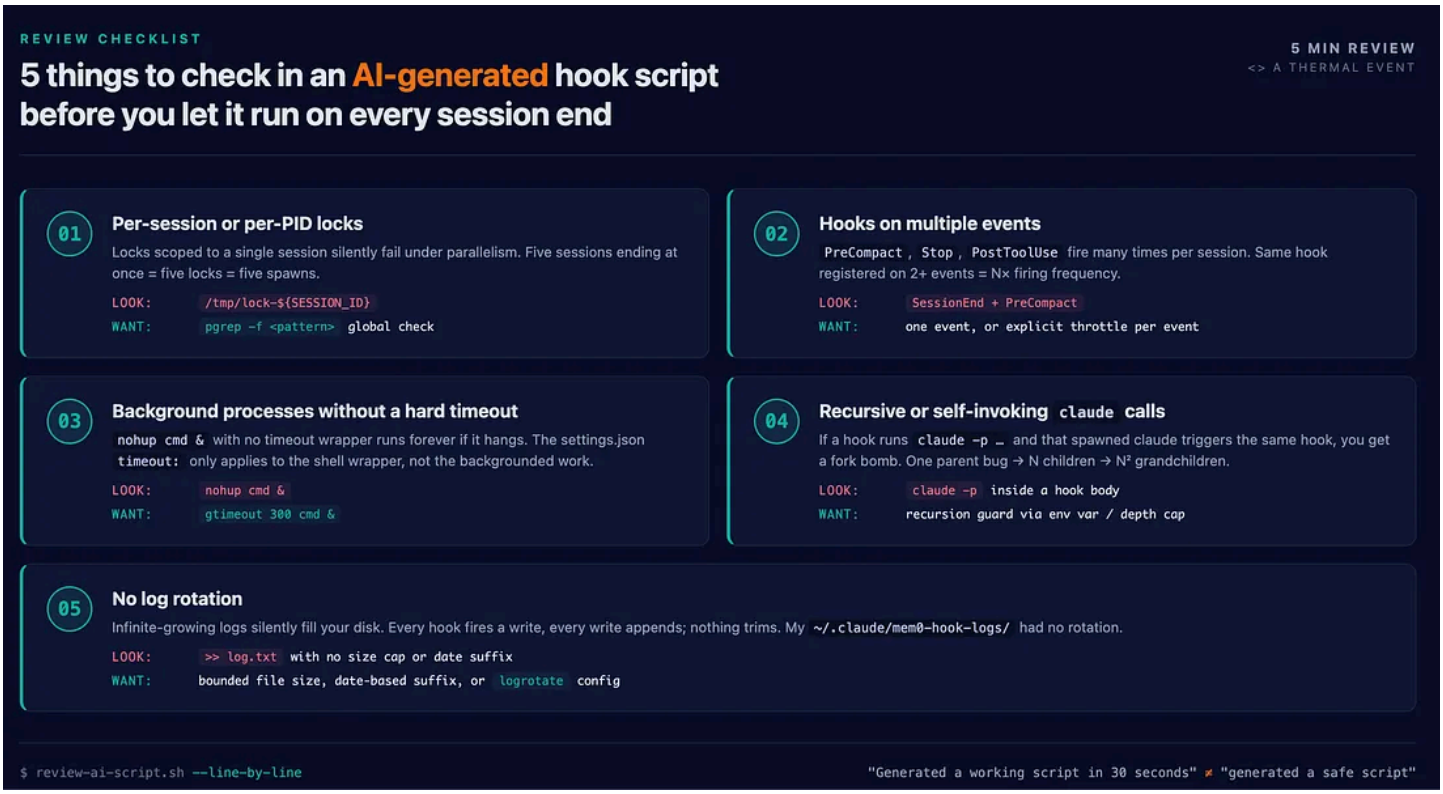
If you’re letting Claude Code install hooks, MCP servers, or anything that spawns background processes on your machine, **read the script line by line before running it.** Specifically look for:

- **Locks that are per-session or per-PID instead of global.** These silently fail under parallelism.
- **Hooks registered on multiple events** without thinking about firing frequency. `PreCompact`, `Stop`, `PostToolUse` — these can fire hundreds of times per session.
- **Background processes without a hard timeout.** `nohup cmd &` with no `timeout` wrapper is a liability.
- **Recursive or self-invoking `claude` calls** inside hooks. One bug in a parent session can spawn N children, each of which can spawn their own.
- **No limit on log files.** Mine were writing to `~/.claude/mem0-hook-logs/` with no rotation.

None of this is a reason not to use Claude Code, or not to let it write hooks. I still use it for exactly this. But **“it generated a working script in 30 seconds” is not the same as “it generated a safe script.”**

Review it like any PR — line by line.

My MacBook is fine. But I spent an afternoon diagnosing an overheating laptop that **a five-minute code review would have prevented.** Don’t be me.



Checklist graphic summarizing the 5 things to look for when reviewing AI-generated hook scripts | Generated by Claude

What This Gets You (When It's Working)

After a week of use, I can do things like:

- "What did I decide about auth?" → gets the x-api-key decision with context
- "Show me all architecture decisions for Drippery" → filtered by project
- "Why did I choose Inngest?" → semantic match, even though I never stored "Inngest" as a keyword

The data persists between sessions, between projects, and between machine reboots. It's just a SQLite file.

What It Doesn't Do

- **Not a replacement for CLAUDE.md** — CLAUDE.md is deterministic (always loaded), Mem0 is probabilistic (search-based). Keep both.
- **Costs tokens** — each mem0_add calls the Anthropic API for memory extraction. The extraction hook adds real cost per session end — budget for it.
- **Cold start** — first tool call loads the embedding model (~3 seconds on M1). Subsequent calls are instant.
- **No project isolation by default** — memories are global. Use the metadata field to tag by project if you need filtering.
- **Will happily overheat your laptop if you let it** — see above.

Try It — Carefully

The full setup takes 15 minutes:

1. `pip install mem0ai chromadb sentence-transformers mcp anthropic`
2. Save the MCP server script to `~/.claude/mem0-mcp-server.py`
3. `claude mcp add mem0 -s user -- python3 ~/.claude/mem0-mcp-server.py`
4. `brew install coreutils` (for `gtimeout`)
5. Write the hook script with a **global** `pgrep`-based concurrency cap and a `gtimeout 300` wrapper
6. Register it on `SessionEnd` only — not `PreCompact`
7. Restart Claude Code
8. **Read the script you pasted in, line by line, before you let it run on every session end**

Your AI coding assistant now has a memory. It's not perfect, but it's **much better than starting from zero every session** — and if you've wired up the guardrails, it won't spawn seven parallel processes behind your back.

Further Reading

- Mem0 — the memory layer used here (GitHub)
- Model Context Protocol — the open standard for Claude Code tools
- ChromaDB — local vector store, SQLite-based
- all-MiniLM-L6-v2 — the embedding model (Hugging Face)
- claude-mem — the alternative I tried first

Generative AI

This story is published on Generative AI. Connect with us on LinkedIn and follow Zeniteq to stay in the loop with the latest AI stories.

Subscribe to our newsletter and YouTube channel to stay updated with the latest news and updates on generative AI. Let's shape the future of AI together!

Programming | Artificial Intelligence | Claude Code | Software Development

Software Engineering

Following

Published in Generative AI

85K followers · Last published 4 hours ago

Stay updated with the latest news, research, and developments in the world of generative AI. We cover everything from AI model updates, comprehensive tutorials, and real-world applications to the broader impact of AI on society. Work with us: jimclydegm@gmail.com

Follow

Written by Daniel Rusnok

739 followers · 48 following

Web FullStack AI Engineer and solo founder. Writing about software, TypeScript, and building Drippery. Dead-simple drip email tool for devs and creators.

No responses yet

Andre Banki

What are your thoughts?